

28 MoE环游记：4、难处应当多投入

Mar By 苏剑林 | 2025-03-28 | 6871位读者

前两篇文章我们都在讨论负载均衡，其中在《MoE环游记：3、换个思路来分配》介绍Loss-Free方案时，笔者留了一个悬念：它引入的Bias项有一个冗余的自由度，这个自由度可以用来做另外有趣的事情。这篇文章我们就来讨论这件事。

我们知道，MoE是为每个Token只选择最匹配的 k 个Expert来进行计算，从而在增大参数量的同时还节省了计算量。然而，当我们仔细思考就会发现，这个策略实际上有明显的可改进之处：直观来看，每个Token的难度并不一样，所以更合理的方案应该是难的Token分配更多的计算资源，简单的token分配更少的资源，这样或许能在同样有限的资源下将效果最大化。

而刚才提到的Bias的额外自由度，恰好可以用来简单地实现这个目标。

设计思想

首先，我们回顾一下，MoE的基本形式是

$$\mathbf{y} = \sum_{i \in \text{argtop}_k \rho} \rho_i \mathbf{e}_i \quad (1)$$

负载不均衡是MoE训练常见的问题，对此研究人员提出了Aux Loss，这部分工作我们介绍于《MoE环游记：2、不患寡而患不均》。此外，在《MoE环游记：3、换个思路来分配》我们介绍了DeepSeek提出的Loss-Free方案，它将MoE改为

$$\mathbf{y} = \sum_{i \in \text{argtop}_k \rho + \mathbf{b}} \rho_i \mathbf{e}_i \quad (2)$$

然后通过调节新引入的Bias项 $\mathbf{b}$ 来实现负载均衡。为了实现每个Token可以选择动态数量的Expert，笔者提出的做法是将Loss-Free的形式稍微修改一下：

$$\mathbf{y} = \sum_{i \in \text{argwhere } \rho + \mathbf{b} > 0} \rho_i \mathbf{e}_i \quad (3)$$

即只要满足 $\rho_i + b_i > 0$ 的Expert就被选中，这样每个Token选出的Expert数量自然是动态的，并且免除了排序的需求，某种程度上看还变得更简化了。

优化目标

$\mathbf{b}$ 的优化目标有两个：一是跟Loss-Free一样，要实现负载均衡；二是要控制每个Token被选中的平均Expert数为 k ，这我们可以称为预算控制，要不然直接 $b_i = \infty$ 将所有Expert都选出来就行了，但这不是我们想要的。

负载均衡依然采样Loss-Free的训练方式。定义记号 $\mathbf{f} = [f_1, f_2, \dots, f_n]$

$$f_i = \begin{cases} 1, & \rho_i + b_i > 0 \\ 0, & \rho_i + b_i \leq 0 \end{cases} \quad (4)$$

然后记 $\tilde{\mathbf{F}} = \mathbb{E}[\mathbf{f}]$ ，那么 $\mathbf{F} = \tilde{\mathbf{F}} / |\tilde{\mathbf{F}}|$ 就是当前Expert分布，其中 $|\tilde{\mathbf{F}}|$ 是 $\tilde{\mathbf{F}}$ 的各分量之和。Loss-Free提出的更新

公式是：

$$\mathbf{b} \leftarrow \mathbf{b} - \alpha \text{sign}(\mathbf{F} - \mathbf{Q}) \quad (5)$$

其中 $\mathbf{Q} = (1/n, 1/n, \dots, 1/n)$ 是目标的均匀分布。我们提到多次， $\mathbf{b}$ 存在一个冗余的自由度，体现在对 $\mathbf{b}$ 所有分量加上同一个常数，排序结果不变。这样一来，我们可以把更新规则(5)改为

$$\mathbf{b} \leftarrow \mathbf{b} - \alpha \left[\text{sign}(\mathbf{F} - \mathbf{Q}) - \overline{\text{sign}(\mathbf{F} - \mathbf{Q})} \right] \quad (6)$$

这里向量上面加一横代表该向量的全体分量的均值，是一个标量，向量减标量代表每个分量都减去这个标量。这样一来出来的 $\mathbf{b}$ 必然满足 $\bar{\mathbf{b}} = 0$ ，但不改变负载均衡的效果。于是我们可以 $\bar{\mathbf{b}}$ 这个自由度留给预算控制。

怎么理解呢？很明显，如果给全体 b_i 都加上同一个正数，那么满足 $\rho_i + b_i > 0$ 的几率将会变大，从而总预算也会增大。所以做法很简单，先算出当前平均预算，不难发现正好是 $|\tilde{\mathbf{F}}|$ ，如果它大于 k ，那么就调小一点 $\mathbf{b}$ ，反之则增大。整合到式(6)是

$$\mathbf{b} \leftarrow \mathbf{b} - \alpha \left[\text{sign}(\mathbf{F} - \mathbf{Q}) - \overline{\text{sign}(\mathbf{F} - \mathbf{Q})} + \text{sign}(|\tilde{\mathbf{F}}| - k) \right] \quad (7)$$

如果只想保证预算不超过 k ，而不非要等于 k ，那么可以改为当 $|\tilde{\mathbf{F}}| < k$ 时不作改变

$$\mathbf{b} \leftarrow \mathbf{b} - \alpha \left[\text{sign}(\mathbf{F} - \mathbf{Q}) - \overline{\text{sign}(\mathbf{F} - \mathbf{Q})} + \text{sign}(\max(|\tilde{\mathbf{F}}| - k, 0)) \right] \quad (8)$$

尝试简化

细细品味式(7)，我们会发现它做了两件事，一是让 $\mathbf{F} = \tilde{\mathbf{F}}/|\tilde{\mathbf{F}}|$ 逼近 $\mathbf{Q}$ ，二是让 $|\tilde{\mathbf{F}}|$ 逼近 k 。这看起来可以合并成一件事：让 $\tilde{\mathbf{F}}$ 逼近 $\tilde{\mathbf{Q}} = k\mathbf{Q} = (k/n, k/n, \dots, k/n)$ 。于是式(7)可以简化为

$$\mathbf{b} \leftarrow \mathbf{b} - \alpha \text{sign}(\tilde{\mathbf{F}} - \tilde{\mathbf{Q}}) \quad (9)$$

笔者将式(7)和式(9)都做了实验，发现它们在效果上大同小异，但是式(9)的负载均衡和预算控制两个指标在训练前期的抖动都大很多，所以追求稳定性的读者可以优先考虑式(7)或(8)，追求简洁的读者则可以考虑式(9)。

考虑到 sign 只保留了 $\tilde{\mathbf{F}}_i - \tilde{\mathbf{Q}}_i$ 的符号而忽略了绝对值的大小，笔者也尝试RMS Norm替代 sign ：

$$\mathbf{b} \leftarrow \mathbf{b} - \alpha(\tilde{\mathbf{F}} - \tilde{\mathbf{Q}})/\|\tilde{\mathbf{F}} - \tilde{\mathbf{Q}}\|_{RMS} \quad (10)$$

其中向量的 $\|\cdot\|_{RMS}$ 是指分量的平方和的平方根。很明显 sign 的RMS是1，而RMS Norm之后RMS也为1，所以两者更新的数量级相同，可以用同一个 α 。由于RMS Norm保留了 $\tilde{\mathbf{F}}_i - \tilde{\mathbf{Q}}_i$ 的相对大小，可以做到误差小的更新也小，所以在波动程度上比 sign 略小，但也好得不多。

当然，用RMS Norm替换 sign 来增加稳定性是一个通用技巧，式(5)、(6)、(7)或(8)都可以做这样的替换，这就看个人审美了，总之只是略稳但不多。

初始方式

解决完 $\mathbf{b}$ 的更新规则，我们来考虑 $\mathbf{b}$ 的初始化，这是一个有意思但不算十分关键的问题。

按照常规做法， b 全零初始化且 ρ 用Sigmoid激活，那么初始阶段会把 n 个Expert都选出来，明显超出 $\leq k$ 的预算，这将会导致非常多的Token Drop。不过，如果我们没有强迫症的话，这并不是很严重的问题，因为模型其他参数通常会加Warmup但 b 通常不加，所以在Warmup的前几步模型就会自动把这个问题解决了。

如果我们介意这一点，那么可以通过调整 b 初始化来控制初始预算。假设Router的输入是 d 维向量，满足零均值、单位方差（有RMSNorm在，近似成立），Router的权重初始化方差为 σ^2 ，那么Router的Logits近似为零均值、 $\sigma^2 d$ 方差。有了这些数据，我们可以用正态近似模拟加二分法估算一个初始 b ：

```

1 import numpy as np
2
3 def sigmoid(x):
4     return 1 / (1 + np.exp(-x))
5
6 def b_init(n, k, d, sigma, eps=0.1):
7     b1, b2 = -1, 0
8     std = sigma * d**0.5
9     logits = np.random.randn(10000, n) * std
10    scores = sigmoid(logits)
11    while True:
12        b = (b1 + b2) * 0.5
13        c = ((scores + b) > 0).sum(1).mean()
14        if -eps < c - k < eps:
15            return b
16        elif c > k:
17            b2 = b
18        else:
19            b1 = b
20
21 b_init(32, 4, 1024, 6e-3)

```

代码中考虑的是Sigmoid激活，所以搜索区间是 $[-1, 0]$ ，如果是其他激活函数请自行调整。不过这里的建议跟《MoE环游记：3、换个思路来分配》一文是相同的，即加 b 的 ρ 可以统一用Sigmoid激活，乘上Expert的 ρ 才考虑用别的激活函数。

相关工作

这篇文章之前，已经有一些工作尝试过动态选择Expert数目的MoE设计，下面简单列举一些笔者搜到的工作，并从个人的审美角度做一些简单的评析。

比较朴素的做法是AdaMoE和MoE++，它们在Expert中混入了一些低计算成本的Expert，如空白Expert、复制Expert、常数Expert，同时也鼓励负载均衡，这样当Token选中这些简单Expert时，等价于少选择了其他标准的Expert，从而间接地实现了动态数目。这样做的好处是可以复用原本Top- k MoE的基建，但同时也欠缺了一些灵活性。

另外一个朴素的想法是将Top- k 选择改为Top- p ，出自《Harder Tasks Need More Experts: Dynamic Routing in MoE Models》。这个转换看上去很自然，但实际上有颇多问题，比如无法准确控制平均预算，因为当 ρ 接近均匀分布时Top- p 的比例会非常大，所以原论文又新增了一项熵损失来让 ρ 远离均匀分布。总的来说，个人感觉它引入的问题比收益更明显。

一个比较独特的做法是Ada-K Routing，它新增一个模块来预测要激活的Expert数，然后用强化学习来训练，这样做在原理上没问题，但引入强化学习无疑会增加训练复杂性。DA-MoE则利用Attention分数来识别重要Token，为其分配更多Expert，但感觉不够本质，因为“MoE”原则上不局限于FFN层，一旦用到Attention上，不就没有Attention分数可用了？

形式上跟本文做法最相似的可能是ReMoE，它同样是基于零阈值来选择Expert，但选择了Aux Loss的方式来实现负载均匀以及预算控制，同时又混合了手搓梯度的思想来控制Aux Loss权重，总体来看多了点糅合感。本文则延续了Loss-Free的思想，利用b的额外自由度来调控这个阈值，从而以最小的改动实现了动态Expert数目。

文章小结

本文提出了一种动态选择Expert数目的MoE设计，主要思想是对Loss-Free的MoE形式稍作修改，然后调整Bias项的更新规则，利用它的额外自由度来同时实现负载均衡和预算控制。

转载到请包括本文地址：<https://spaces.ac.cn/archives/10815>

更详细的转载事宜请参考：《科学空间FAQ》

如果您需要引用本文，请参考：

苏剑林. (Mar. 28, 2025). 《MoE环游记：4、难处应当多投入 》 [Blog post]. Retrieved from <https://spaces.ac.cn/archives/10815>

```
@online{kexuefm-10815,
  title={MoE环游记：4、难处应当多投入},
  author={苏剑林},
  year={2025},
  month={Mar},
  url={\url{https://spaces.ac.cn/archives/10815}},
}
```